

RISC-V 32-bit CPU Design

발표자: 서윤철

목차

- RISC란?
- 개발 환경
- RISC-V Instruction Set
- 개발한 하드웨어 구조
- Program Test
- Verification
- Trouble Shooting
- 고찰

RISC란? *Reduced Instruction Set Computer*

프로젝트 목표: RV32I CPU 설계

RISC-V 32-bit Instruction set

vs ARM, x86과의 차이

x86	Intel · AMD CPU에 쓰이는 복잡한 CISC ISA, 연식이 된 제품과 호환이 좋음.
ARM	가장 널리 사용됨 but 라이선스 비용 발생.
RISC-V	오픈, 무료, 확장성 뛰어남.

HDL

SystemVerilog

개발 환경

VS Code

검증 환경

Vivado VCS Verdi

개발 진행

Single-Cycle
CPU



Multi-Cycle
CPU

Multi-Cycle 기준으로 설명...

Instruction Set

- type별 명령어 정리

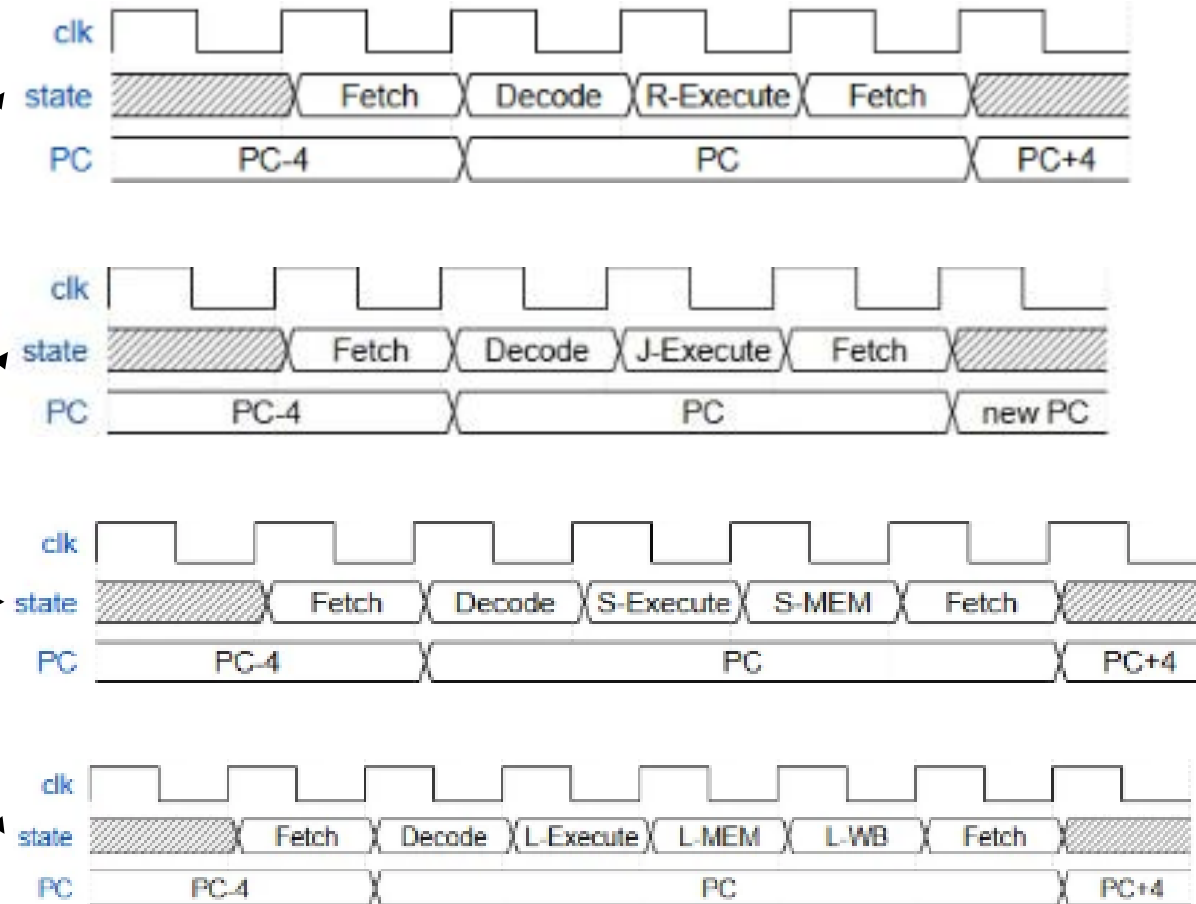
type	opcode	Instruction
R	0110011	ADD,SUB,SLL,SRL,SRA,SLT,SLTU,XOR,OR,AND
L	0000011	LB,LH,LW,LBU,LHU
I	0010011	ADDI,SLTI,SLTIU,XORI,ORI,ANDI,SLLI,SRLI,SRAI
S	0100011	SB,SH,SW
B	1100011	BEQ,BNE,BLT,BGE,BLTU,BGEU
LU	0110111	LUI
AU	0010111	AUIPC
J	1101111	JAL
JL	1100111	JALR

Instruction Set

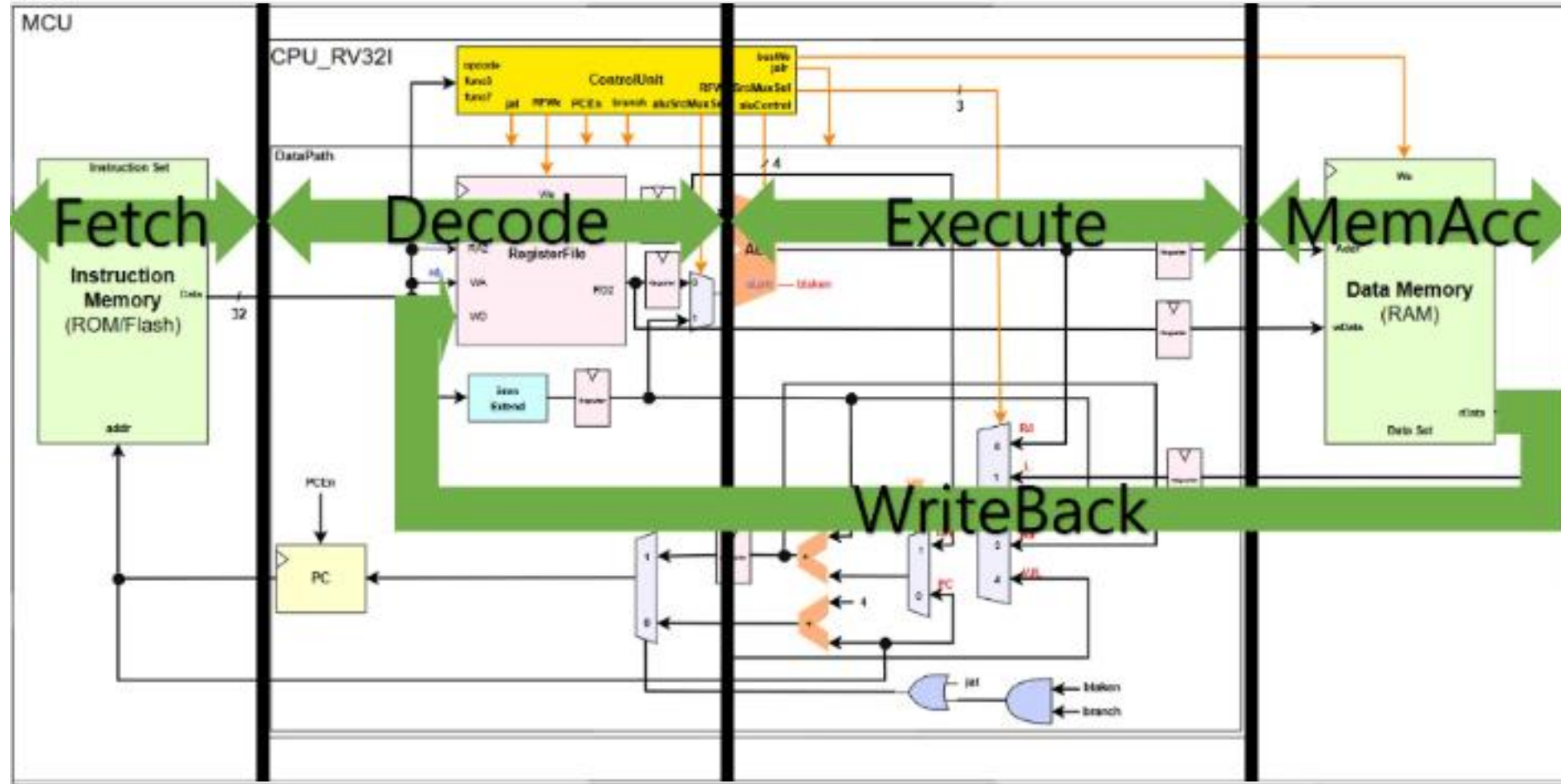
- type별 상태 진행 정리

type	State Transition
R	Fetch -> Decode -> Execute
I	
B	
LU	
AU	
J	
JL	
S	Fetch -> Decode -> Execute -> Memory Access
L	Fetch -> Decode -> Execute -> Memory Access -> Write Back

- Timing Diagram

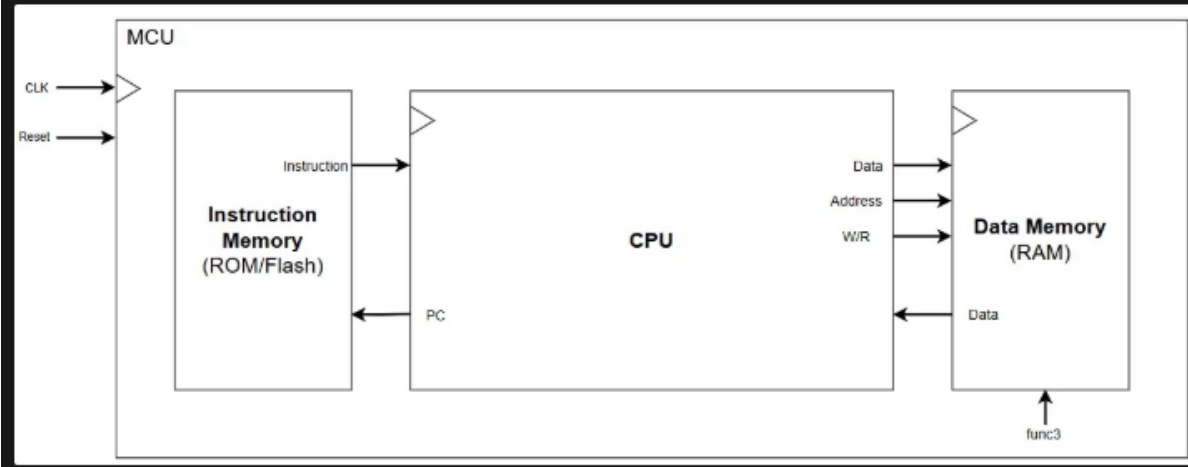


Diagram

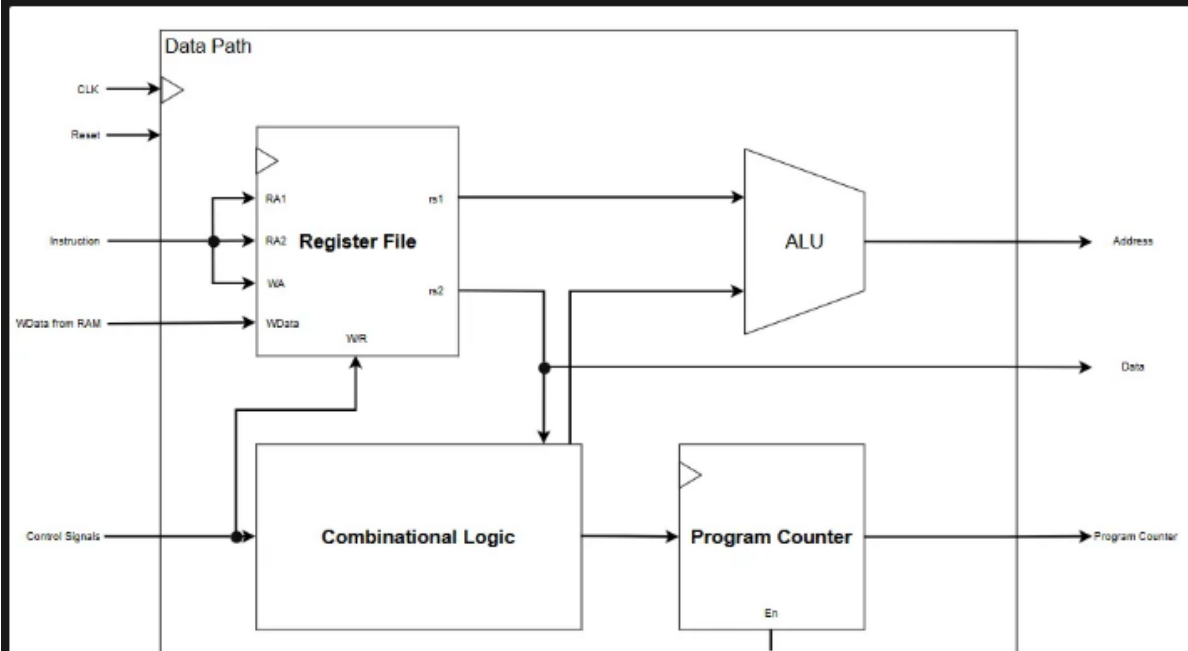


개발한 하드웨어 구조

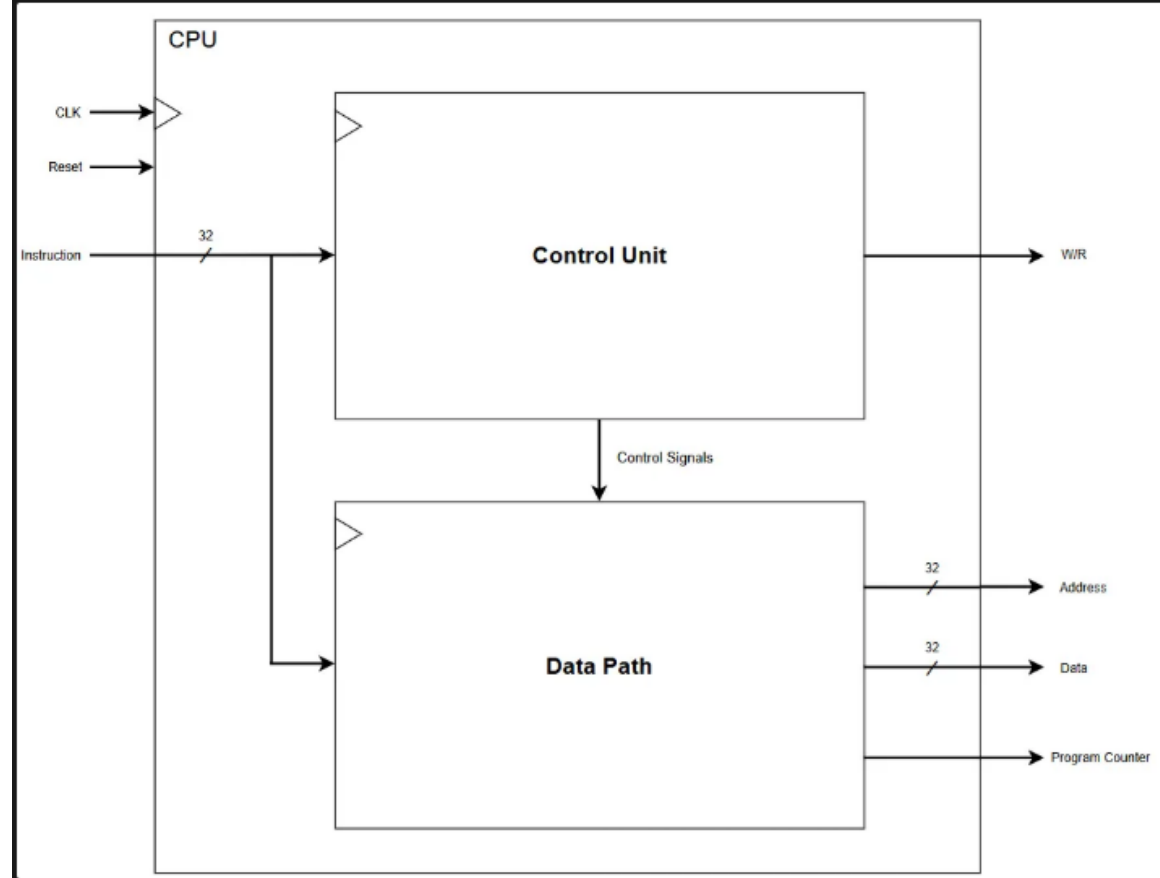
• MCU(TOP module)



• Data Path



• CPU



Test Program

```
int is_prime(int x);

int main(void) {
    int prime = 0;

    for (int i=11;i<=20;i++) {
        if (is_prime(i)) prime++;
    }

    return 0;
}

int is_prime(int x) {
    if (x < 2) return 0;

    for (int d=2; d<x; d++) {
        int t = x;
        while (t >= d) {
            t = t - d;
        }
        if (t == 0) return 0; // 나머지=0 → 나누어 떨어짐
    }
    return 1;
}
```

<Prime Number Counting Program>

prime number(소수): 1보다 큰 자연수 중에서 1과 자기 자신만을 약수로 가지는 수

- Main

구간(11~20) 내의 숫자를 비교하여 카운트.

- int is_prime(int x)

정수 값을 받고 그 값이 소수이면 1을 반환,
소수가 아니면 0을 반환.

CPU 검증을 위한 프로그램이므로, prime number를 찾기 위해 나누기 연산 대신,
뺄셈 연산을 사용.

Test Program

```
int is_prime(int x);

int main(void) {
    int prime = 0;

    for (int i=11; i<=20; i++) {
        if (is_prime(i)) prime++;
    }

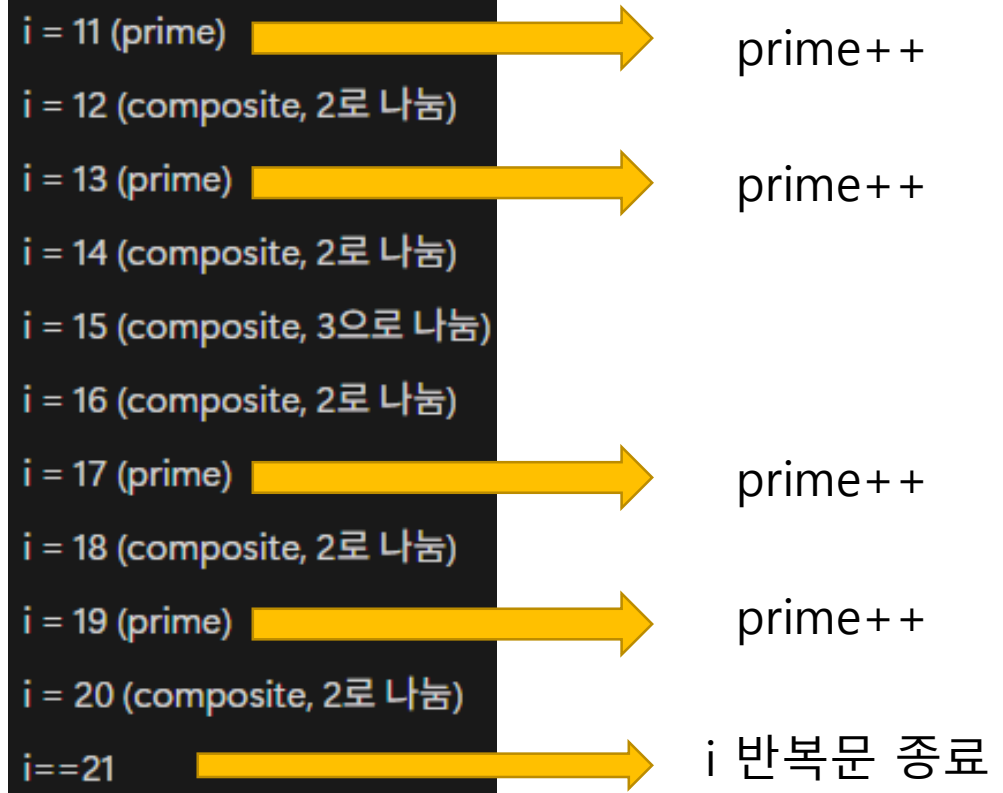
    return 0;
}

int is_prime(int x) {
    if (x < 2) return 0;

    for (int d=2; d<x; d++) {
        int t = x;
        while (t >= d) {
            t = t - d;
        }
        if (t == 0) return 0; // 나머지=0 → 나누어 떨어짐
    }
    return 1;
}
```

<Prime Number Counting Program>

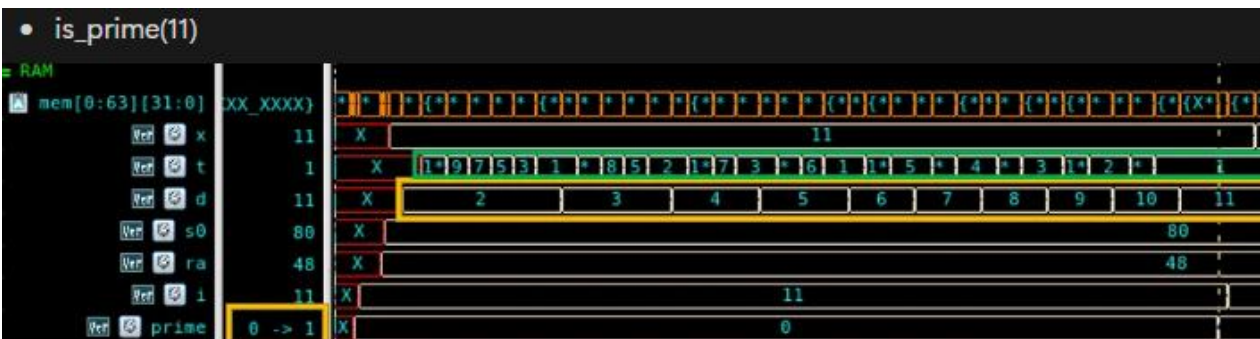
- 예상되는 시나리오



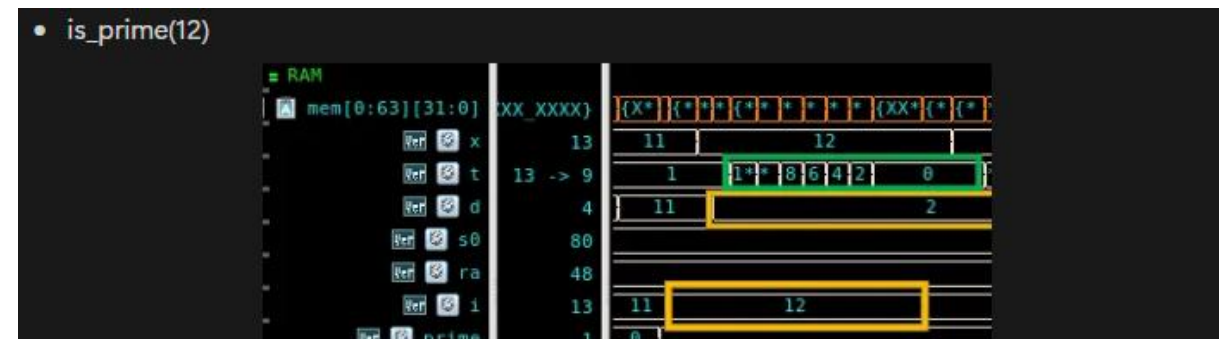
Test Program

<Simulation>

<Prime Number Counting Program>



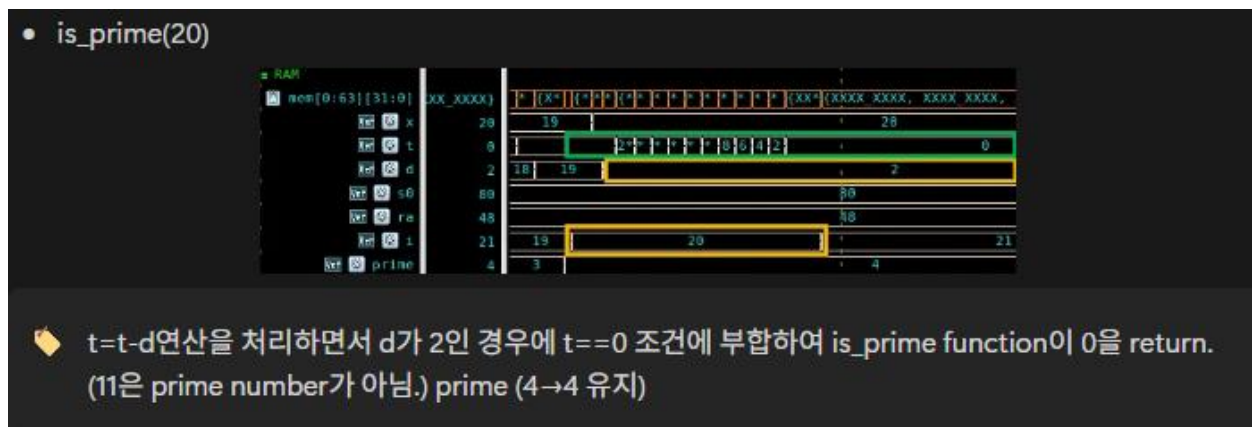
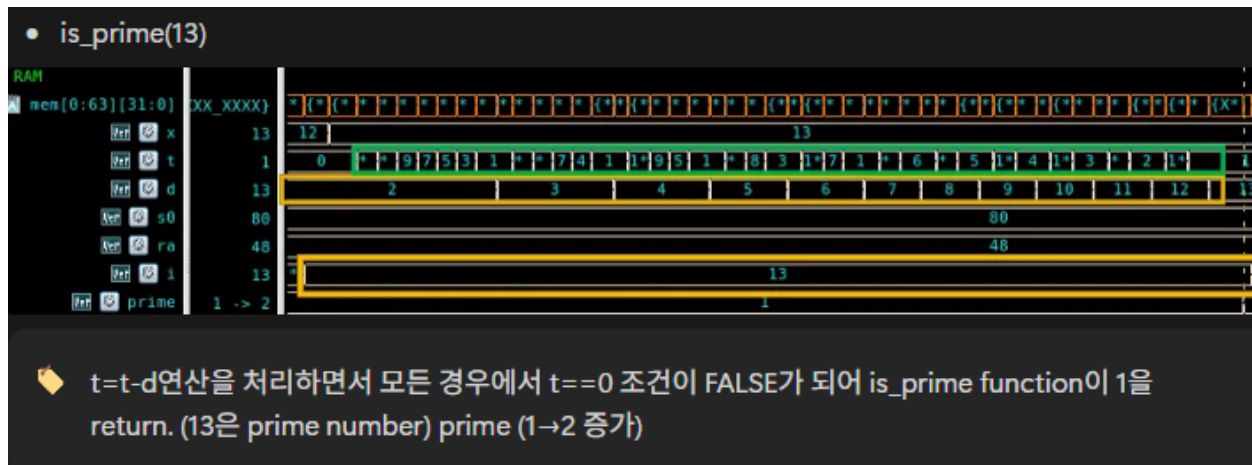
◆ t=t-d연산을 처리하면서 모든 경우에서 t==0 조건이 FALSE가 되어 is_prime function이 1을 return. (11은 prime number) prime (0→1 증가)



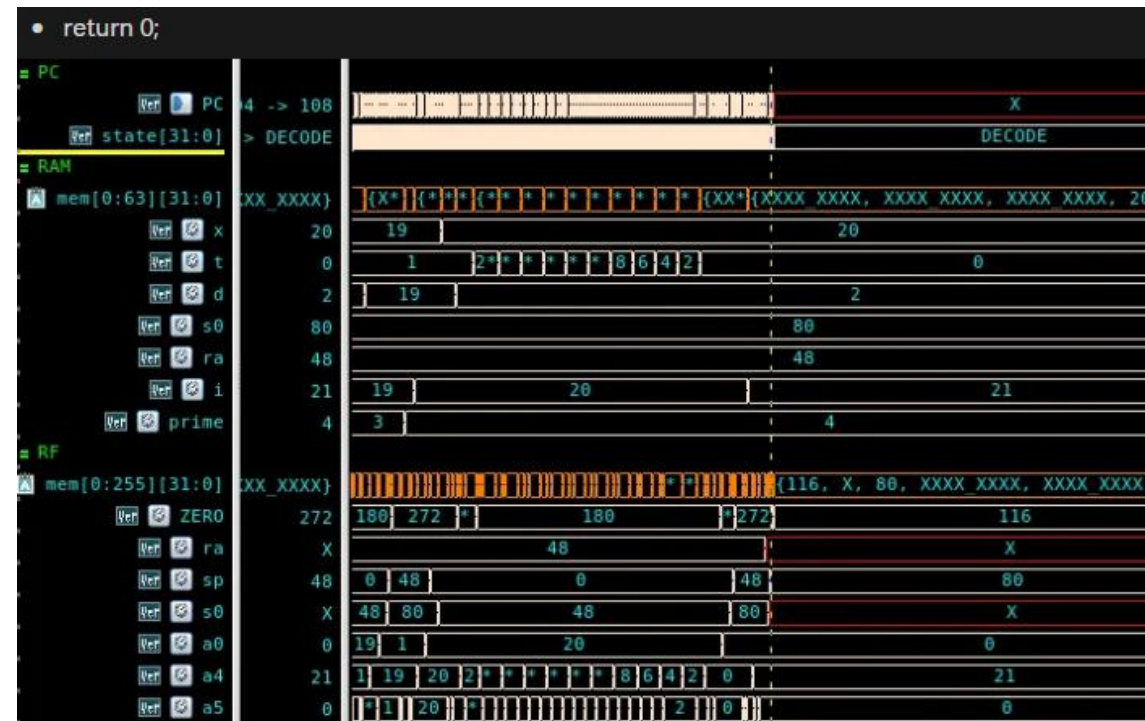
◆ t=t-d연산을 처리하면서 d가 2인 경우에 t==0 조건에 부합하여 is_prime function이 0을 return. (11은 prime number가 아님.) prime (1→1 유지)

Test Program

<Simulation>



<Prime Number Counting Program>



• result

i	11	12	13	14	15	16	17	18	19	20
prime number	O	X	O	X	X	X	O	X	O	X

prime: 4

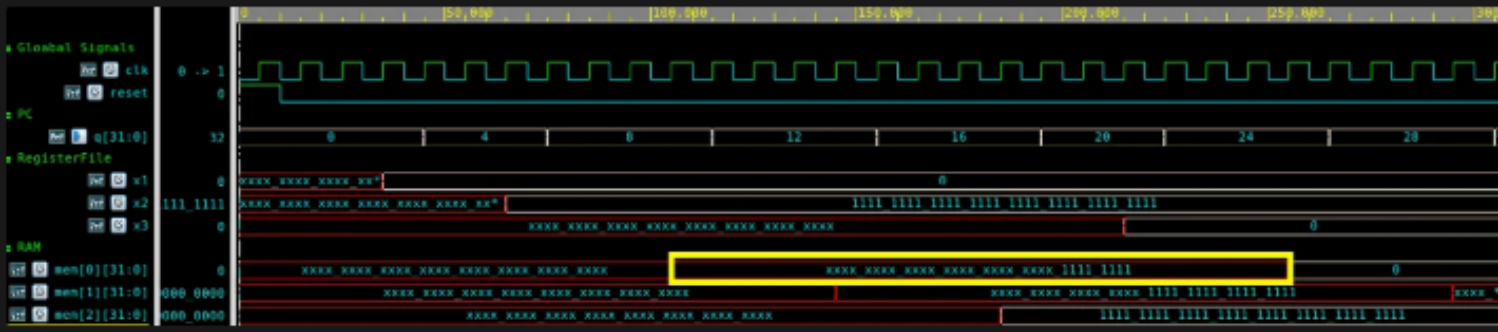
Verification

<Simulation>

- SB, SH, SW

추가 내용

- sb, sw

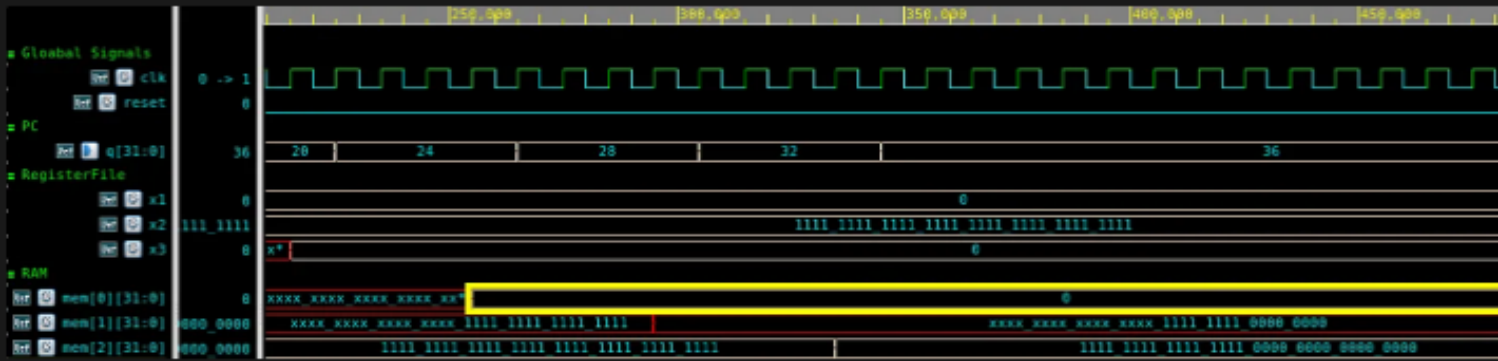


sb x2, 0(x1) → mem[0] = x2(32b'1111_1111_1111_1111_1111_1111 = -32'd1)
 : 하위 8bit만 store된 모습
 32'bxxxx_xxxx_xxxx_xxxx_xxxx_xxxx_xxxx_xxxx → 32'bxxxx_xxxx_xxxx_xxxx_xxxx_xxxx_1111_1111

MNEMONIC	NAME	Descript
SB	Store Byte	M[rs1+imm][0:7] = rs2[0:7]
SH	Store Half	M[rs1+imm][0:15] = rs2[0:15]
SW	Store Word	M[rs1+imm][0:31] = rs2[0:31]

```

addi x1, x0, 0
addi x2, x0, -1
sb x2, 0(x1)
sh x2, 4(x1)
sw x2, 8(x1)
addi x3, x0, 0
sw x3, 0(x1)
sb x3, 4(x1)
sh x3, 8(x1)
    
```



sw x3, 0(x1) → mem[0] = x3(32'd0)
 : 전체 bit가 0으로 된 모습
 32'bxxxx_xxxx_xxxx_xxxx_xxxx_xxxx_1111_1111 → 32'b0

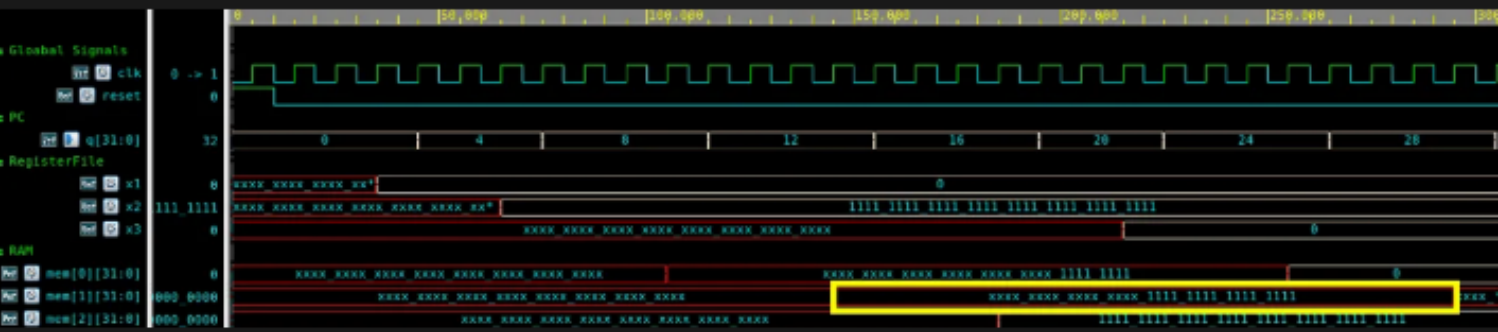
Verification

<Simulation>

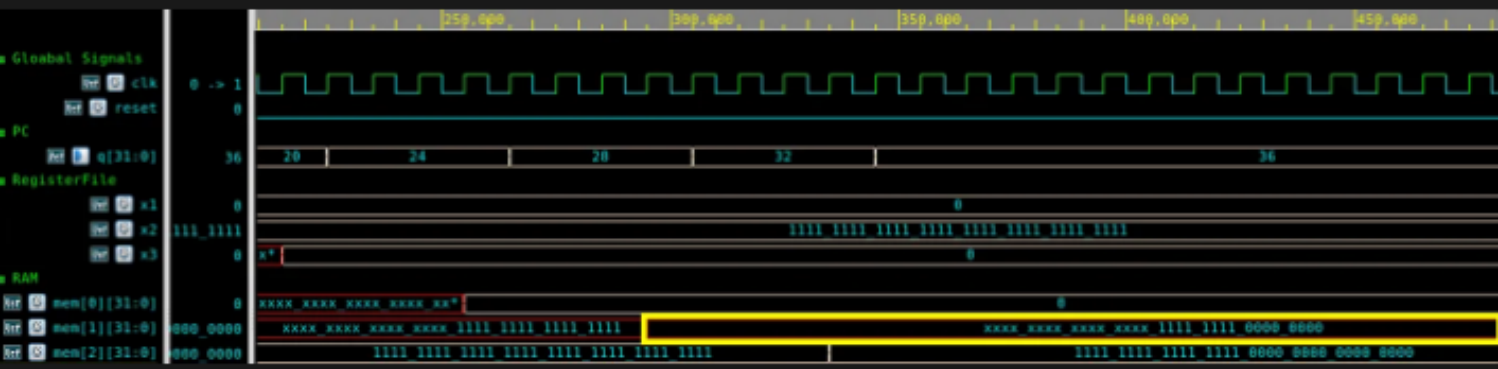
- SB, SH, SW

추가 내용

- sh, sb



sh x2, 4(x1) → mem[(0+4)/4] = x2(32'd1)
: 하위 16bit만 store된 모습
32'bxxxx_xxxx_xxxx_xxxx_xxxx_xxxx_xxxx_xxxx → 32'bxxxx_xxxx_xxxx_xxxx_1111_1111_1111_1111



sb x3, 4(x1) → mem[(0+4)/4] = x3(32'd0)
: 하위 8bit만 store된 모습
32'bxxxx_xxxx_xxxx_xxxx_1111_1111_1111_1111 → 32'bxxxx_xxxx_xxxx_xxxx_1111_1111_0000_0000

MNEMONIC	NAME	Descript
SB	Store Byte	M[rs1+imm][0:7] = rs2[0:7]
SH	Store Half	M[rs1+imm][0:15] = rs2[0:15]
SW	Store Word	M[rs1+imm][0:31] = rs2[0:31]

```
addi x1, x0, 0
addi x2, x0, -1
sh x2, 0(x1)
sh x2, 4(x1)
sw x2, 8(x1)
addi x3, x0, 0
sw x3, 0(x1)
sb x3, 4(x1)
sh x3, 8(x1)
```

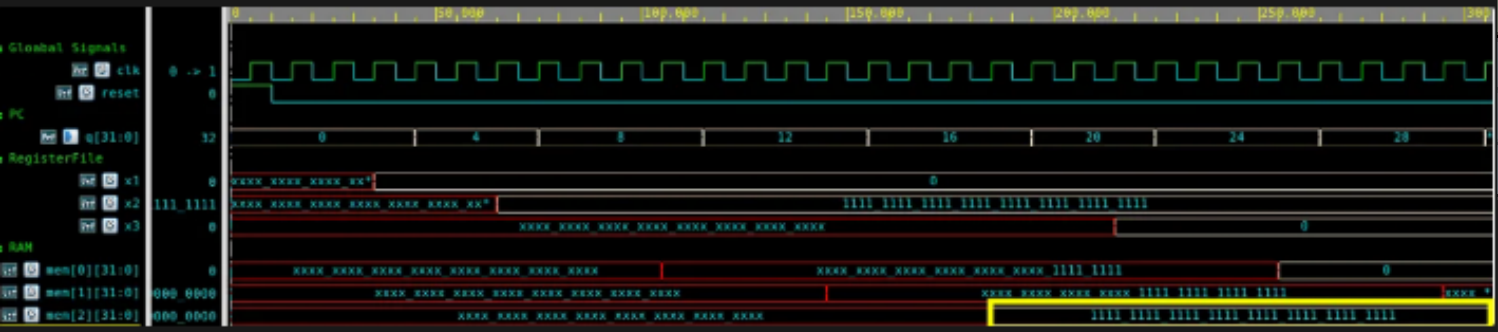
Verification

<Simulation>

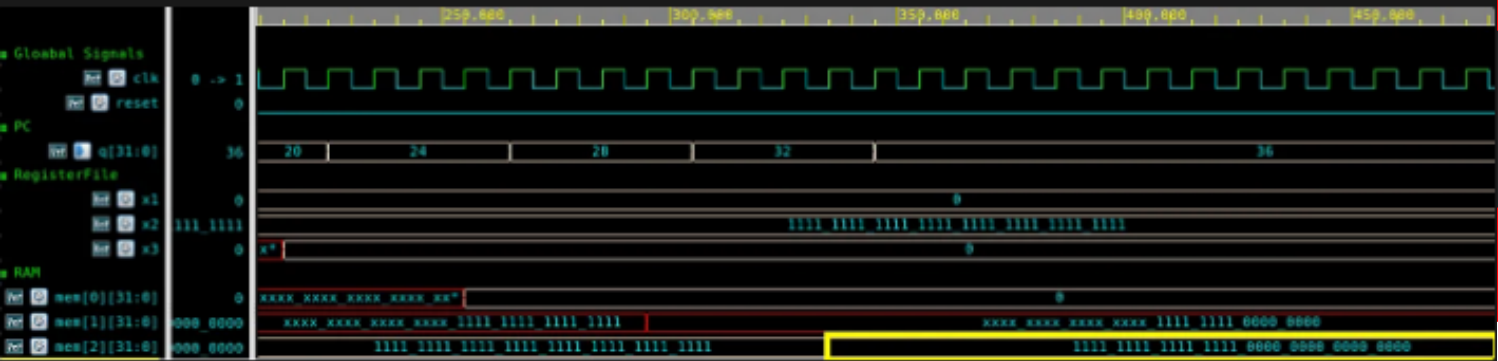
- SB, SH, SW

추가 내용

- sw, sh



sw x2, 8(x1) —→ mem[(0+4)/4] = x2(-32'd1)
: 전체 bit에 store된 모습
32'bxxxx_xxxx_xxxx_xxxx_xxxx_xxxx_xxx → 32'b1111_1111_1111_1111_1111_1111_1111



sh x3, 8(x1) —→ mem[(0+8)/4] = x3(32'd0)
: 하위 16bit만 store된 모습
32'b1111_1111_1111_1111_1111_1111_1111 → 32'b1111_1111_1111_1111_0000_0000_0000_0000

MNEMONIC	NAME	Descript
SB	Store Byte	M[rs1+imm][0:7] = rs2[0:7]
SH	Store Half	M[rs1+imm][0:15] = rs2[0:15]
SW	Store Word	M[rs1+imm][0:31] = rs2[0:31]

```
addi x1, x0, 0
addi x2, x0, -1
sb x2, 0(x1)
sh x2, 4(x1)
sw x2, 8(x1)
addi x3, x0, 0
sw x3, 0(x1)
sb x3, 4(x1)
sh x3, 8(x1)
```

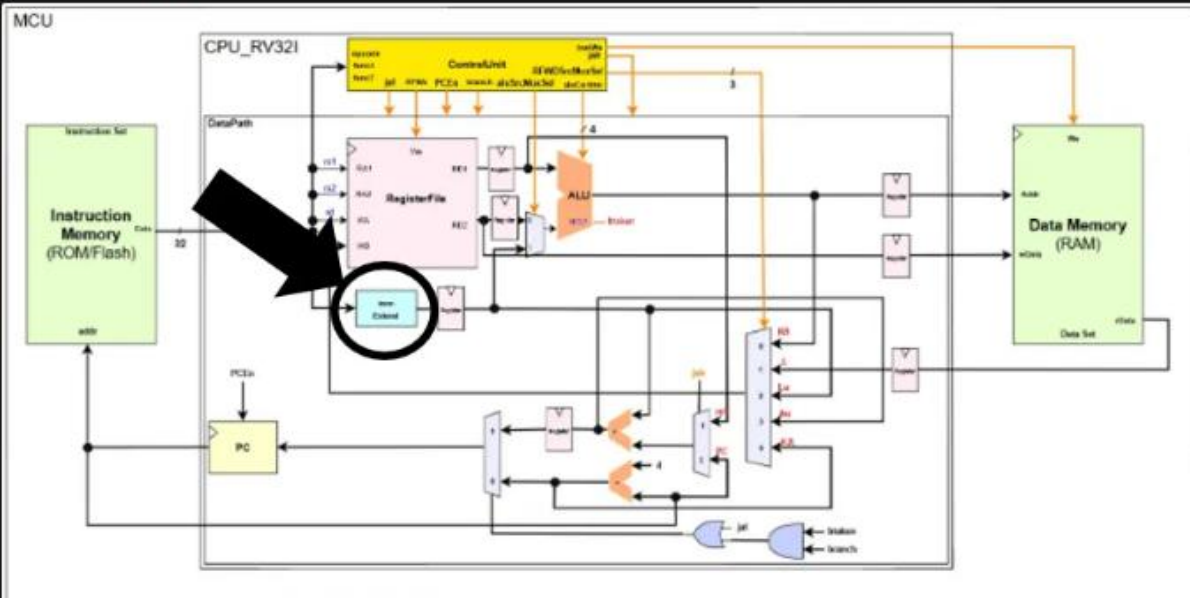

Trouble Shooting

Instruction Set에 대한 이해 부족 1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE	MNEMONIC
immediate (20)																				Register Destination (5)					Opcode (7)								
imm[31:12]																				rd					0 1 1 0 1 1 1							LU-TYPE	LUI
imm[31:12]																				rd					0 0 1 0 1 1 1							AU-TYPE	AUIPC

RISC-V Instruction Set 타입 중 LUI와 AU를 구현하기 위해 직접 shift하는 과정을 추가하기 위해 모듈을 추가하여 불필요한 하드웨어를 추가함.

→ imm을 처리하는 모듈에서 한꺼번에 처리하도록 수정



stack pointer의 초기 값 선정

수업 시간에 다루었던 C코드의 assembly코드에 사용했던 stack pointer의 초기 값을 그대로 로드하여 시뮬레이션 중간에 CPU의 동작이 멈추는 현상 관측. assembly코드를 분석한 결과, (addi sp, sp, -32)같은 branch명령어 후 stack pointer 값을 연산하는 과정에서 sp값이 음수가 될 수 있는 조건을 파악. → sp값이 음수가 되는 경우를 방지하고자 sp의 더 높은 초기 값을 로드하여 해결.

후기

- 이번 프로젝트를 통해 CPU 구조를 하드웨어 관점에서 이해할 수 있는 계기가 된 것 같다.

- 단순히 하드웨어만을 구현하는 것을 넘어서 C 코드를 assembly 코드로 바꾸고, assembly 코드를 32-bit 명령어 바꾸는 과정을 경험해보고, assembly 코드를 분석해보면서 CPU 구조에 대해 고민할 수 있는 시간이 된 듯하다.

- 수업이 참 좋았어요.

- 교수님 참 좋아요.



감사합니다.